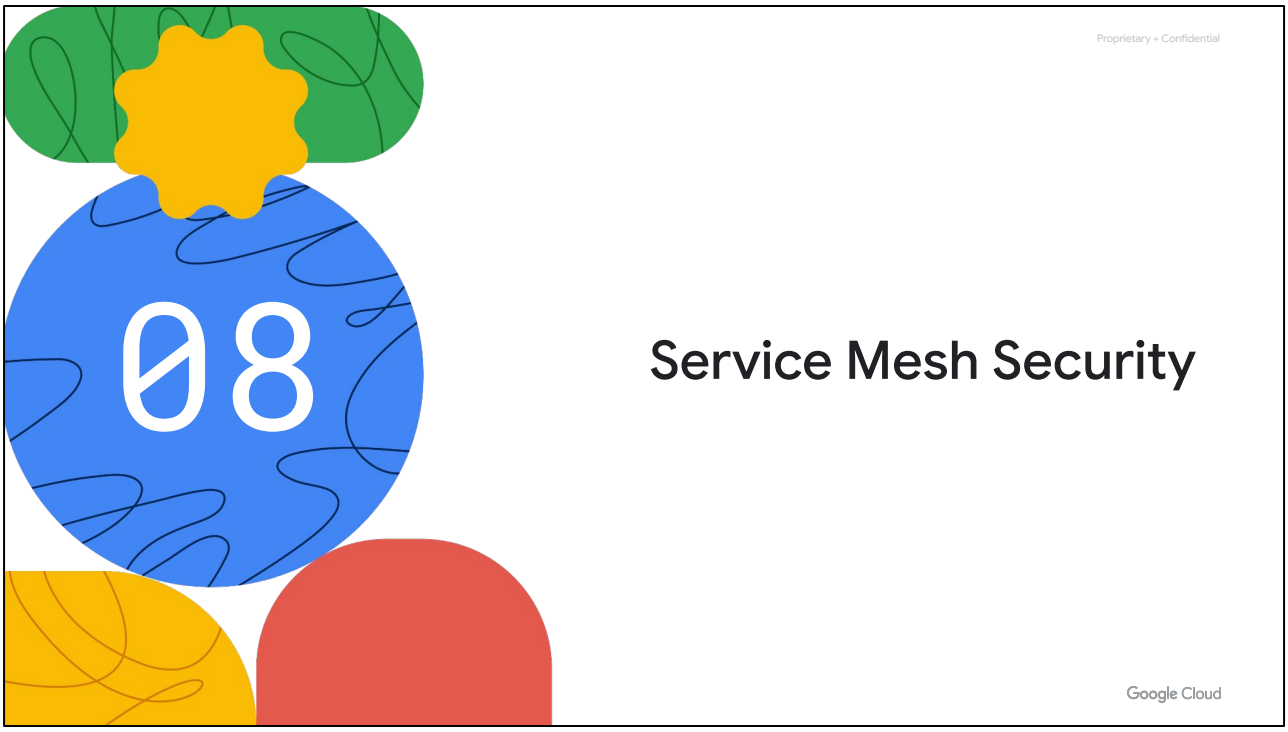


Microservices offer improved agility, scalability, and reusability, but also introduce security challenges, like:

- Network traffic read by unintended services.
- Increased risk of impersonation attacks, where one service might masquerade as another to intercept or manipulate communication.
- Unregulated access, creation, or modification of private or confidential information.
- Service disruptions from denial-of-service attacks.

Network security goals help solve these challenges.

- Encrypt microservice traffic to restrict information access.
- Authenticate services to verify other services and establish trust.
- Authorize services, so that information can only be accessed by allowed services.
- Optionally, add additional access limitations such as quotas to control the load.

Proprietary + Confidential

Service Mesh Security

01 Authentication and encryption

02 Service authentication in the mesh

03 End-user authentication in Cloud Service Mesh

04 Authorization in Cloud Service Mesh

Google Cloud

In this section, titled “Service Mesh Security,” you learn to:

- Encrypt traffic between microservices to prevent anyone in the network from gaining access to private information.
- Authorize services and requests, ensuring that services only access the information that is allowed access from other services.
- Authenticate and authorize services and requests to verify trust among services in the mesh and among end users.
- Limit service access in the network so that granular controls over the communication can be established.

Proprietary + Confidential

Service Mesh Security

01 Authentication and encryption

02 Service authentication in the mesh

03 End-user authentication in Cloud Service Mesh

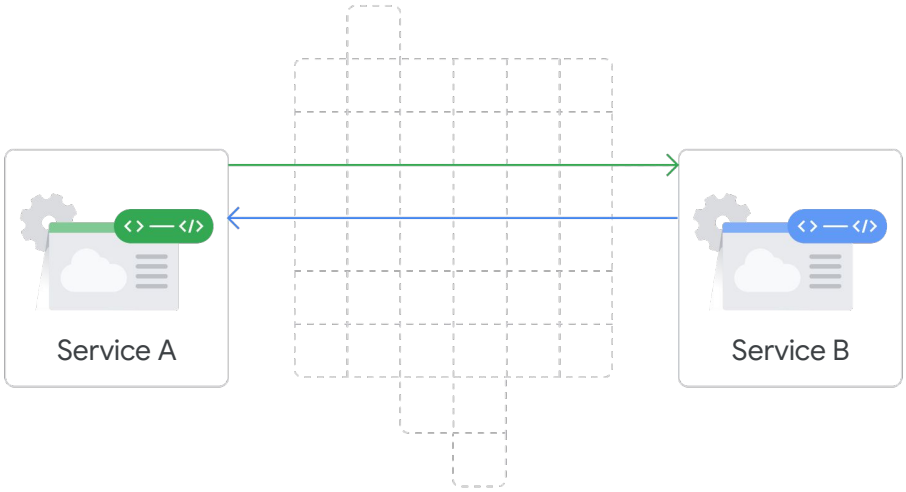
04 Authorization in Cloud Service Mesh

Google Cloud

Let’s explore authentication and encryption with an example.

Proprietary + Confidential

Service A transfers data to Service B over a network

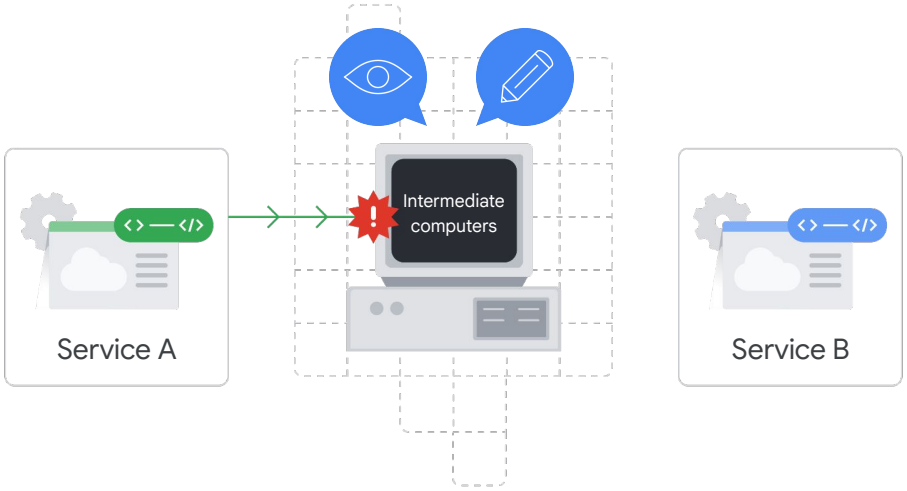


Google Cloud

In this example, Service A communicates with Service B by transferring data over a network.

Proprietary + Confidential

Intermediaries can read or modify data

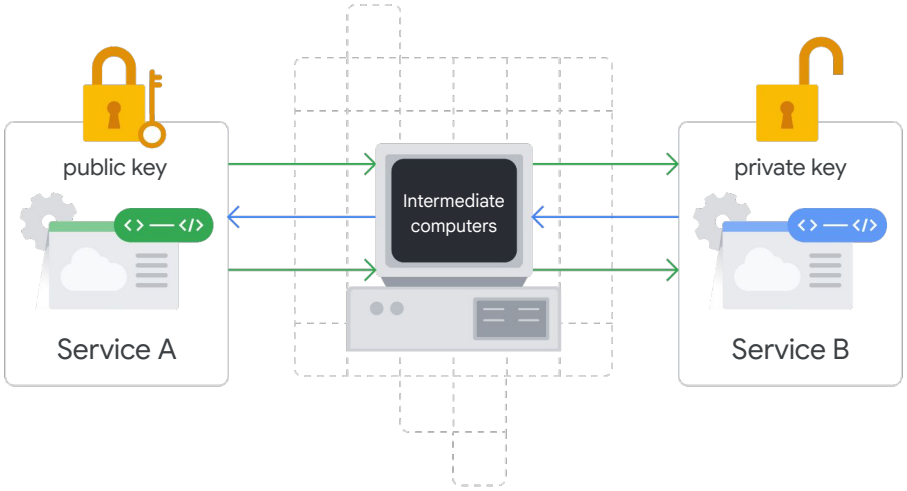


Google Cloud

Because communication is indirect, intermediaries like routers, switches, or other servers have the ability to read or even modify the data, and Service A can't validate that Service B received the request as intended.

Proprietary + Confidential

Asymmetric encryption with Transport Layer Security



Google Cloud

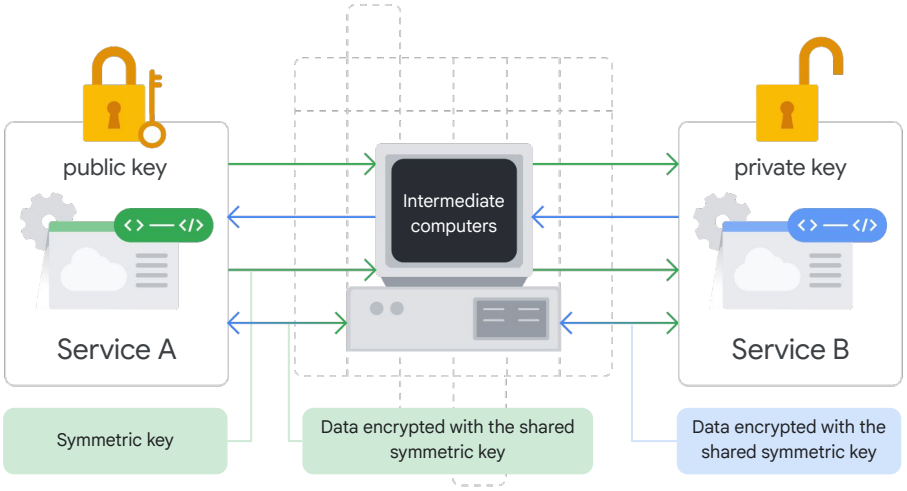
To stop intermediaries from reading and modifying data, it can be encrypted with the Transport Layer Security protocol, or TLS.

This process uses two keys: a public key to encrypt data, and a private key to decrypt data.

Server A uses the public key from Server B to encrypt the message. Then Server B decrypts the message with its own private key. This process is called **asymmetric encryption**.

Proprietary + Confidential

Microservices use symmetric encryption



Google Cloud

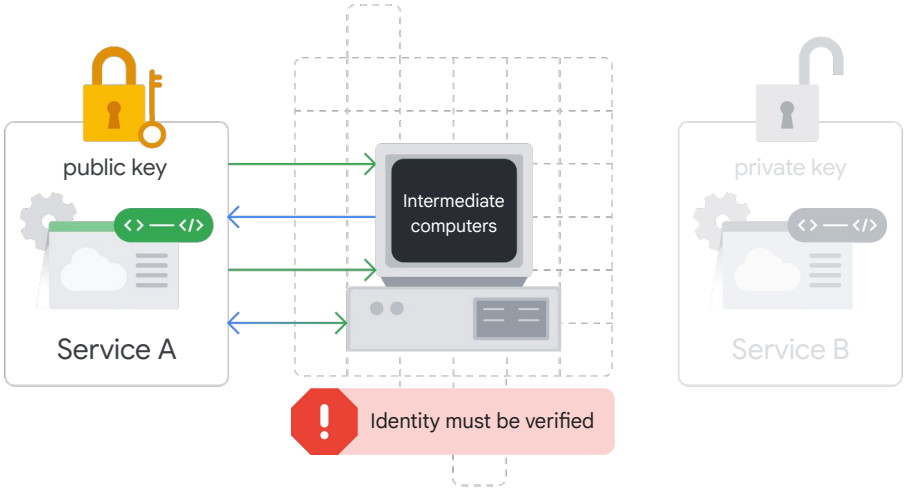
In a microservice architecture, services communicate in both directions, so traffic from Service B back to Service A must also be encrypted.

To achieve this, Service A shares a symmetric key with Service B when it establishes the connection, which can be used by both services to encrypt and decrypt messages.

The only services with access to the symmetric key are Services A and B, so the data cannot be encrypted by others.

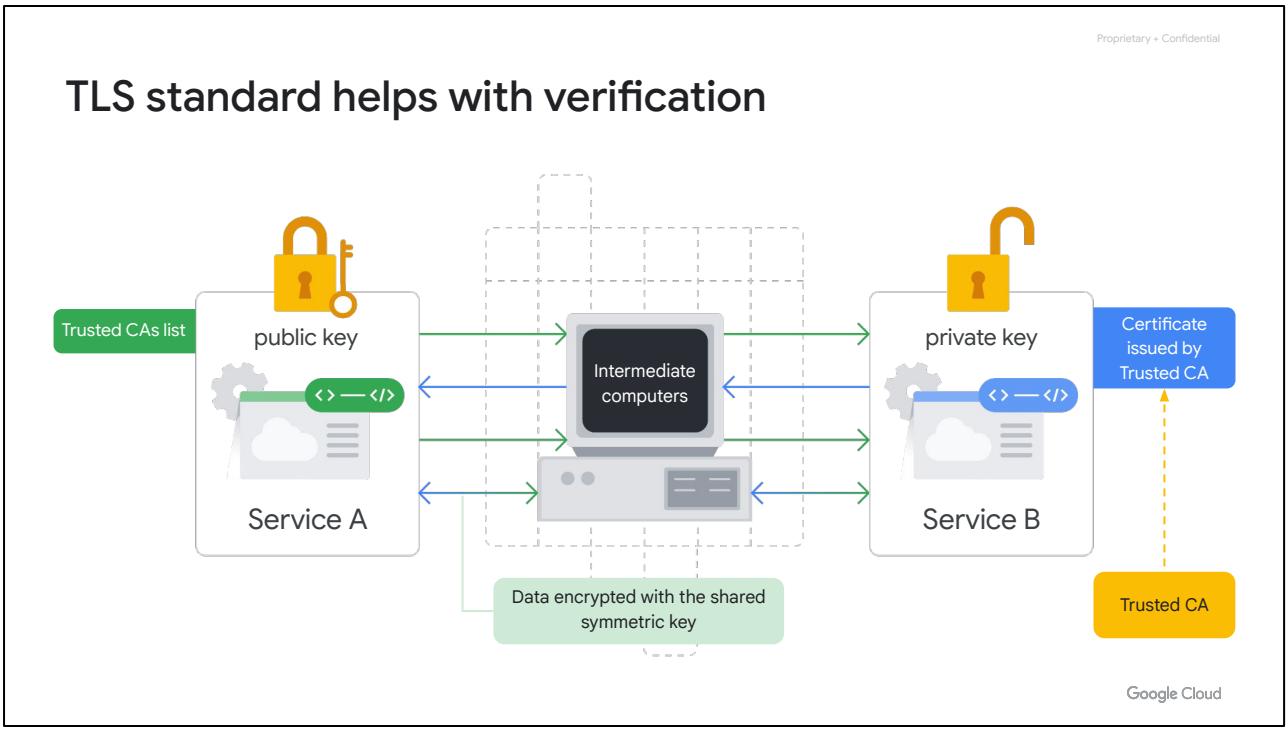
Proprietary + Confidential

Intermediary claims to be Service B



Google Cloud

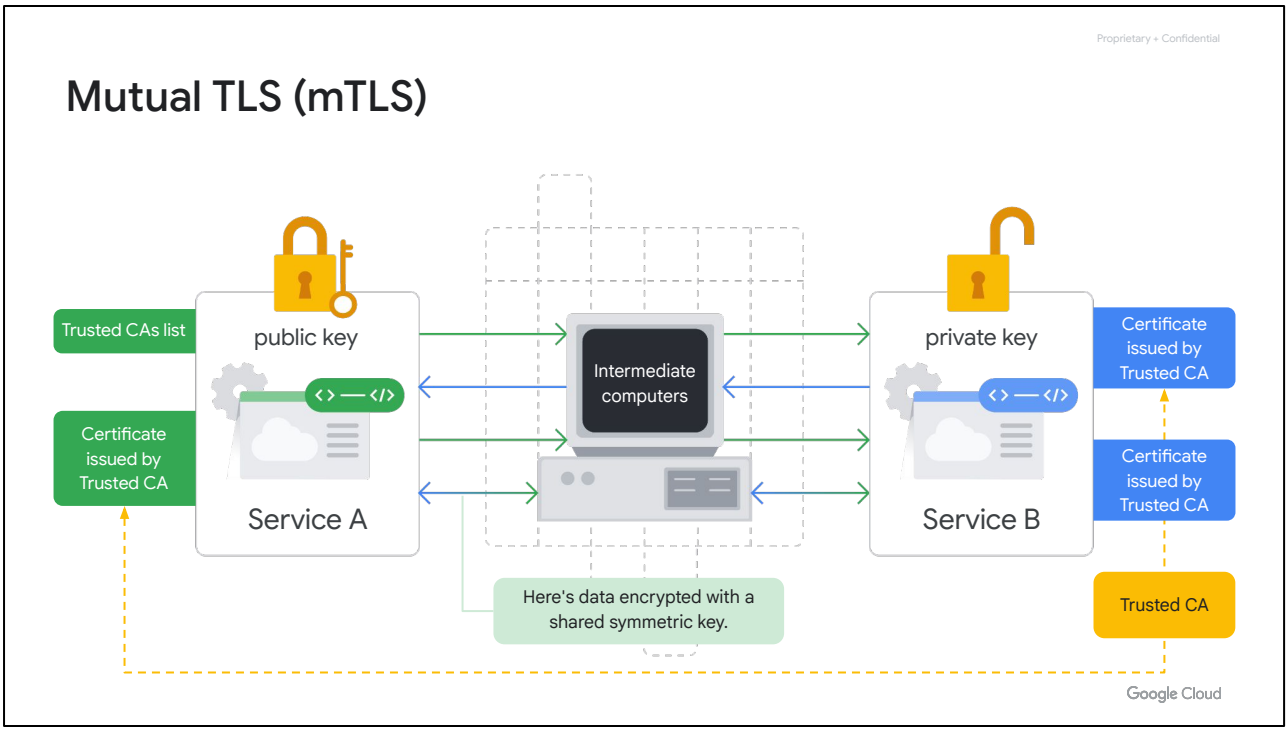
But what if an intermediate computer intercepts the traffic and claims to be Service B? The identity of service B must be verified before it shares a public key.



TLS standard can help solve this problem.

When Service A first tries to establish an encrypted connection, it not only shares a symmetric key but also asks Service B for a certificate of its identity, in the form of X.509.

A certificate authority (CA) is like the trusted advisor of the requester service, in this case Service A. CA validates the certificate provided by Service B and determines whether or not to proceed. This is the same process used by websites that run over HTTPS.



When Service A wants to establish a connection with Service B, both certificates are exchanged and checked by the trusted CA. The symmetric key is used to encrypt the messages.

The process of authenticating services and encrypting messages in both directions is called mutual TLS or mTLS.

In a microservices architecture, managing certificates and mTLS for a large number of services can be complex. Service meshes like Istio simplify this process by providing automated certificate management and mTLS enforcement.

Proprietary + Confidential

Service Mesh Security

01 Authentication and encryption

02 Service authentication in the mesh

03 End-user authentication in Cloud Service Mesh

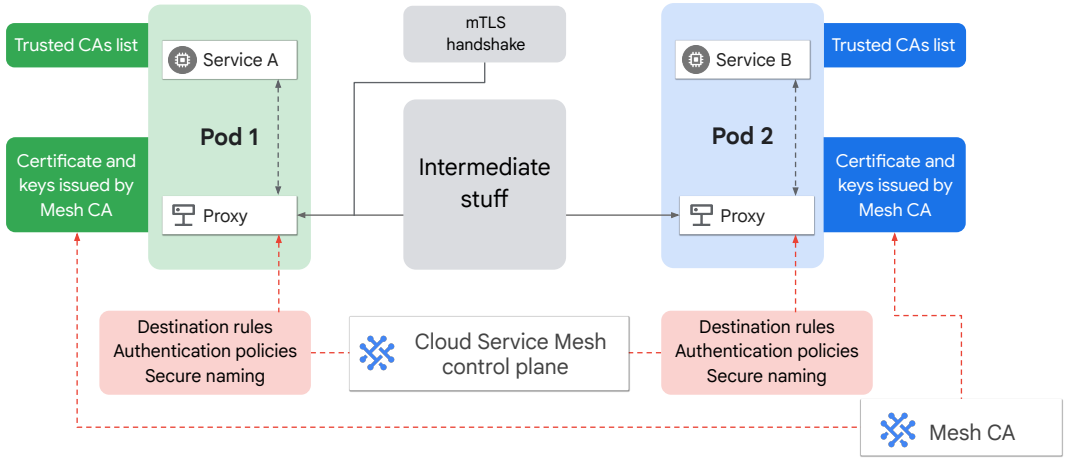
04 Authorization in Cloud Service Mesh

Google Cloud

Cloud Service Mesh offers a **managed CA** and tools to control the communication protocol.

Proprietary + Confidential

Mesh CA



Google Cloud

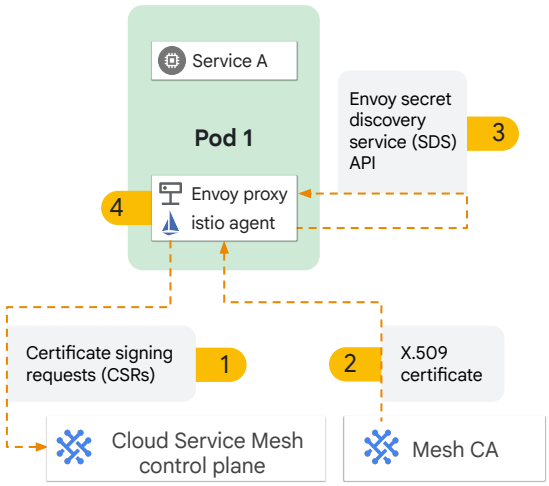
The certificate authority in a service mesh is called Mesh CA, and it's responsible for distributing, rotating, and managing certificates to provide identity.
When pods are created, Mesh CA shares certificates and keys that are later used to verify identity in the network.

Administrators control the communication protocol using destination rules and authentication policies. These policies are applied to the sidecar proxies and act as policy enforcement endpoints (PEPs) to secure communication between the client and the server.

Then traffic between services in the mesh can be secured with mTLS.

Proprietary + Confidential

The Mesh CA provisioning process



Certificate provisioning flow:

1. On pod creation, the istio agent issues a certificate signing request (CSR) to the control plane.
2. Mesh CA sends the X.509 certificates to the istio agent.
3. The istio agent sends the certificates and the private key to Envoy via the Envoy SDS API.
4. The istio agent monitors the expiration of the certificate, and the process repeats periodically for certificate and key rotation.

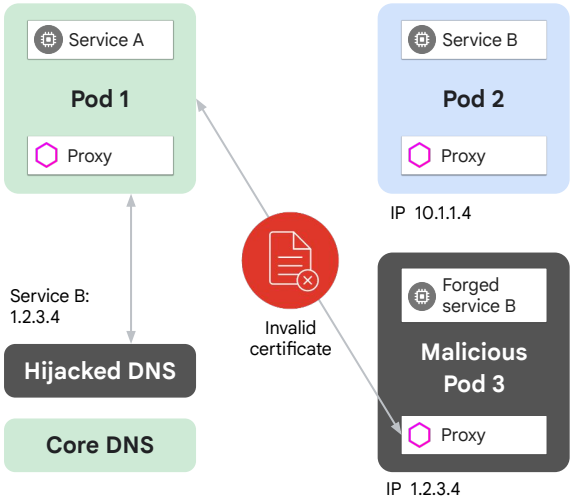
Google Cloud

The Mesh CA provisioning process consists of four steps.

1. First, when a pod is created, a sidecar proxy is injected into the pod containing an Envoy proxy and an Istio agent. The Istio agent communicates with the control plane over gRPC and issues a certificate signing request (CSR) so that the control plane can communicate with Mesh CA.
2. Then, Mesh CA sends the X.509 certificates to the Istio agent.
3. The Istio agent uses the Envoy SDS API to send the certificates and the private key to Envoy.
4. Finally, the Istio agent monitors the expiration of the certificate. The process repeats periodically for certificate and key rotation.

Proprietary + Confidential

Secure naming prevents spoofing attacks



Google Cloud

To confirm that the server is authorized to run the workload, its identity needs to be verified by Envoy using a feature called secure naming.

Say a malicious user intercepts traffic to Service B and redirects it to a fake server.

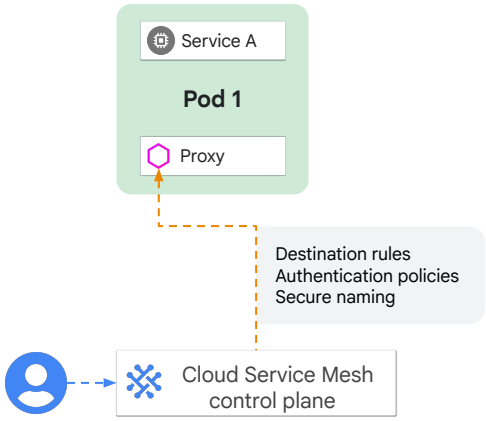
In this example, the user interfered with the DNS service.

We see a compromised DNS service, but an attacker could have also compromised the BGP or ARP protocol.

Because Pod 3's certificate cannot be verified by the proxy's secure naming feature, any connection attempts from other services will be blocked.

Proprietary + Confidential

Admins specify the protocol used in the mesh



Destination rules are applied to specific services and dictate protocol to be used: TLS, mTLS or plain text.

Peer authentication policies are applied mesh-wide or to a specific namespace or workload and enforce a protocol to be used.

Google Cloud

Administrators can use destination rules and peer authentication policies to specify the protocol used in the mesh.

Destination rules offer granular control over communication by allowing you to specify the protocol (TLS, mTLS, or plaintext) used for connections to specific services.

Peer authentication policies provide broader control, enabling you to enforce protocols across an entire mesh, namespace, or specific workload.

Proprietary + Confidential

Destination rules offer granular control

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: mtls-backend
spec:
  targets:
  - name: backend.prod.svc.cluster.loc
  trafficPolicy:
  - tls:
      mode: ISTIO_MUTUAL
```

Destination rule options:

- **ISTIO_MUTUAL**: Establish secure connections to the upstream service using mutual TLS with client certificate authentication.
- **MUTUAL**: Same option as ISTIO_MUTUAL but with custom certificates.
- **SIMPLE**: Originates a TLS connection to the upstream endpoint.
- **DISABLE**: Does not set up a TLS connection to the upstream endpoint.

Google Cloud

You can choose from several **Destination Rule** options.

- ISTIO_MUTUAL: Establish secure connections to the upstream service using mutual TLS with client certificate authentication.
- MUTUAL: It is the same as ISTIO_MUTUAL but with custom certificates.
- SIMPLE: It originates a TLS connection to the upstream endpoint.
- DISABLE: It does not set up a TLS connection to the upstream endpoint.

Proprietary + Confidential

Peer authentication determine the level of TLS

```yaml
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: mtls-backend
  namespace: secure
spec:
  selector:
    matchLabels:
      app: backend
  mtls:
    mode: STRICT
```

Workload-specific policies are specified with a namespace and a selector.

Namespace-specific policies have a namespace but not a selector.

Mesh-wide policies don't have a namespace, but they can have a selector.

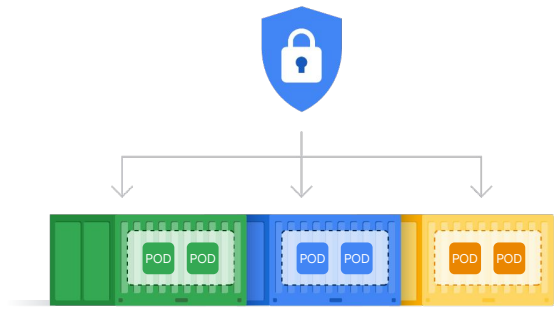
Google Cloud

Peer authentication policies determine the level of TLS in the mesh:

- Workload-specific policies are specified with a namespace and a selector.
- Namespace-specific policies have a namespace but not a selector.
- Mesh-wide policies don't have a namespace, but they can have a selector.

Proprietary + Confidential

Mesh-wide policies



Use cases:

- Setting default security policies for the entire mesh.
- Enforcing global authentication requirements.
- Providing a fallback mechanism when no namespace-specific or workload-specific policies are defined.

Google Cloud

Mesh-wide policies apply to all workloads across all namespaces in your service mesh.

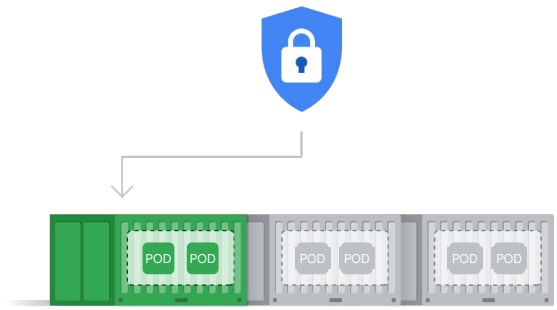
Use cases on why we would use such a broad policy include:

- Setting default security policies for the entire mesh.
- Enforcing global authentication requirements.
- Providing a fallback mechanism when no namespace-specific or workload-specific policies are defined.

For example, you might want to enable permissive mutual TLS mesh-wide, allowing services to gradually adopt stricter authentication.

Proprietary + Confidential

Namespace-specific policies



Use cases:

- Establishing a common security baseline for services within a namespace.
- Simplifying policy management when all services in a namespace share similar security requirements.

Google Cloud

Then there are namespace-specific policies, which apply to all workloads within a specific namespace.

Use cases include:

- Establishing a common security baseline for services within a namespace.
- Simplifying policy management when all services in a namespace share similar security requirements.

You could have a namespace-wide policy for your internal services where mutual TLS is strictly enforced, while another namespace for external-facing services might have more relaxed authentication.

Proprietary + Confidential

Workload-specific policies

Use cases:

- Fine-grained control over authentication requirements for a particular service.
- Applying different authentication policies to different versions of the same service (e.g., canary deployments).
- Isolating specific workloads with stricter security requirements.

Google Cloud

These policies are the most specific. They apply to individual workloads (pods) within a namespace.

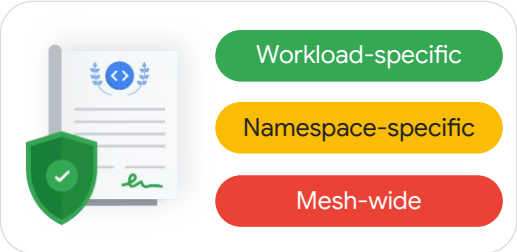
These policies have a very narrow scope and are ideal for:

- Fine-grained control over authentication requirements for a particular service.
- Use of different authentication policies for different versions of the same service (e.g., canary deployments).
- Isolation of specific workloads with stricter security requirements.

For example, you might have a workload running a payment service that requires stronger authentication (e.g., client certificate validation) than other workloads in the same namespace.

Proprietary + Confidential

Policy priority and scoping



- The workload-specific policy will take priority, followed by namespace-specific, and finally mesh-wide.
- If a single level has multiple policies, the oldest policy will take effect.
- By strategically combining these policy scopes, you can create a layered security model.

Google Cloud

If there are conflicts across policies, the workload-specific policy will take priority, followed by namespace-specific, and finally mesh-wide.

If a single level has multiple policies, for example, two policies that span across the mesh, the oldest policy will take effect.

By strategically combining these policy scopes, you can create a layered security model that effectively protects your services in Cloud Service Mesh.

Proprietary + Confidential

Specify the level of TLS enforcement

`mode: STRICT`	Mandates mTLS for all communication.
`mode: PERMISSIVE`	Automatically negotiates the highest level of TLS supported by all communicating clients.
`mode: DISABLE`	Turns off TLS encryption entirely.
`mode: UNSET`	Inherit from parent.

Google Cloud

You can also specify the level of TLS enforcement:

- STRICT: mandates mTLS for all communication. Both the client and server must present valid certificates to establish a connection. If a workload can't use mTLS, communication will be blocked.
- PERMISSIVE: Automatically negotiates the highest level of TLS supported by all communicating clients, allowing for connections ranging from plaintext to full mTLS. This is ideal for gradually introducing mTLS into your environment, as it allows teams to enable TLS on their workloads at their own pace.
- DISABLE: turns off TLS encryption entirely. This leaves communication vulnerable to eavesdropping and tampering and is **strongly discouraged** except in very specific testing or debugging scenarios.
- UNSET: Inherit from parent, if has one. Otherwise treated as PERMISSIVE.

Proprietary + Confidential

Service Mesh Security

01 Authentication and encryption

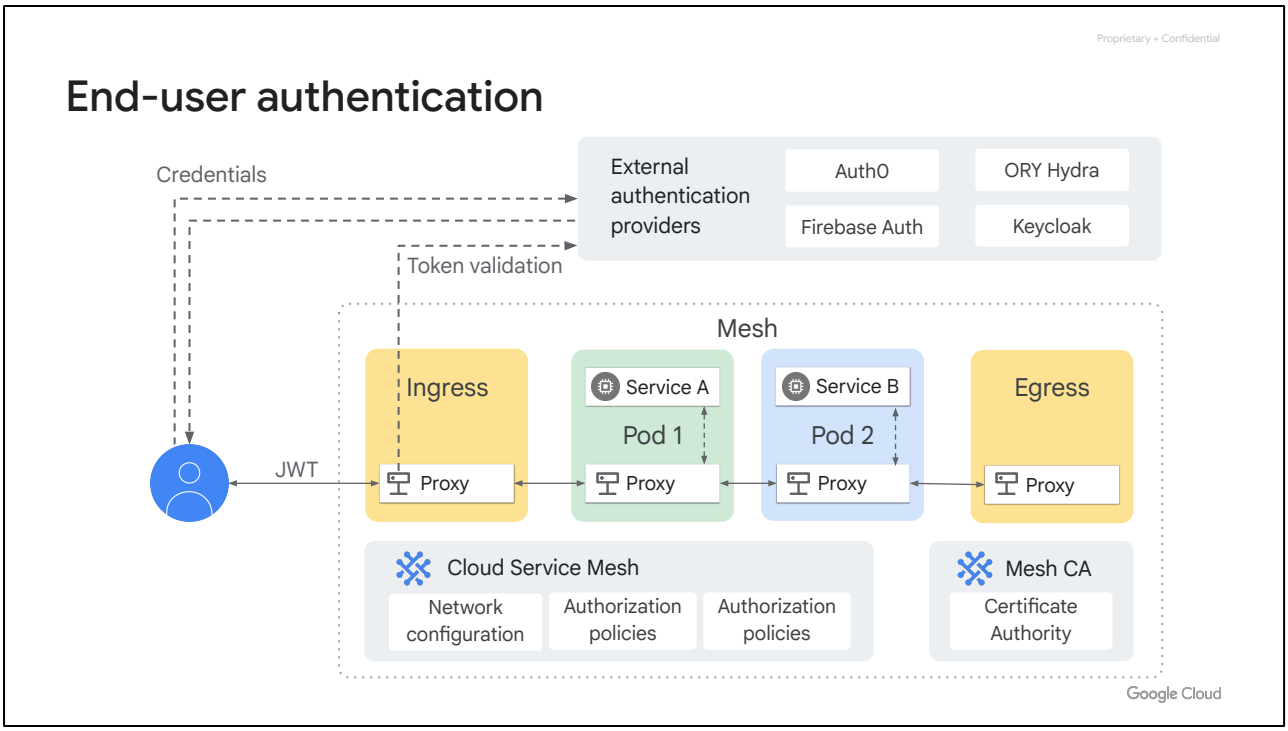
02 Service authentication in the mesh

03 End-user authentication in Cloud Service Mesh

04 Authorization in Cloud Service Mesh

Google Cloud

In addition to service authentication, you can also authenticate individual user requests. This allows you to verify the identity of end-users accessing your applications.



Requests are authenticated with JSON Web Tokens (JWT).

End-users can authenticate using a variety of methods, including custom authentication providers or popular OpenID Connect providers like Auth0, ORY Hydra, Firebase Authentication, or Keycloak.

The provider returns the token, which can be used to place requests to services in the mesh. Before authentication, the token is validated by the gateway's sidecar proxy.

Request authentication policies verify identity

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: httpbin
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
  jwtRules:
  - issuer: “issuer-foo”
    jwksURI:
https://example.com/jwks.json
```

Values that must be verified:

- The location
- The issuer
- The public JSON Web Key Set

Google Cloud

JWTs use request authentication policies to specify values that must be verified, including:

- The location of the token in the request.
- The issuer of the request.
- The public JSON Web Key Set (JWKS) used to verify the token's signature.

If the authentication information provided in the request is invalid, it’s rejected.

Request authentication policies verify identity

```
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: httpbin
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
  jwtRules:
  - issuer: “issuer-foo”
    jwksURI:
https://example.com/jwks.json
```

- Requests without identity are accepted but won't be authenticated.
- JWTs from different providers are acceptable, but requests can only have one valid token.

Google Cloud

Requests without identity are accepted but won't be authenticated.

JWTs from different providers are acceptable, but requests can only have one valid token.

If more than one token is provided, the output principle of the request is undefined.

Proprietary + Confidential

Authservice reduced management overhead

- Authservice handles use authentication with JWTs and intercepts incoming requests.
- It implements the client-side of the OIDC protocol, allowing users to authenticate via a browser using an interactive flow to establish a short-lived session.
- It uses industry standard protocols to integrate with any OIDC authorization server.

Google Cloud

Cloud Service Mesh's authservice component reduces management overhead by handling user authentication with JWTs.

Acting as an external authorization service within the Envoy-based ingress, it intercepts incoming requests.

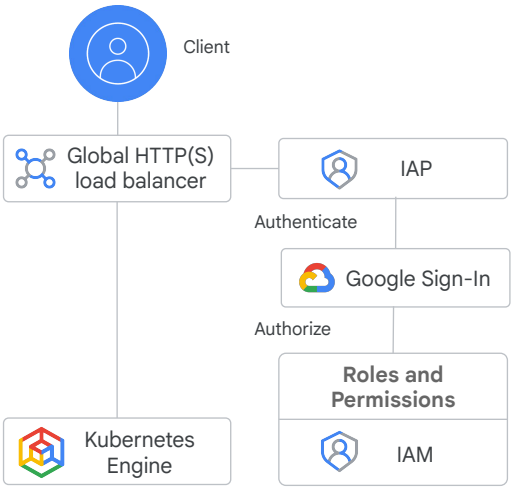
Cloud Service Mesh's authservice implements the client-side of the OIDC protocol, allowing users to authenticate via a browser using an interactive flow to establish a short-lived session.

It uses industry standard protocols to integrate with any OIDC authorization server.

Proprietary + Confidential

Identity-Aware Proxy (IAP)

1. Requests come through Cloud Load Balancing (HTTPS) or internal HTTP load balancing.
2. Request headers or cookies are sent to IAP.
3. If no credentials exist, requests are redirected to OAuth 2.0 Google Account sign-in flow.
4. If requests are valid, the identity is obtained.
5. IAP applies the relevant IAM policy to check whether the user is authorized to access the requested resource.



Google Cloud

When a request goes to the Cloud Load Balancer (HTTPS) or internal HTTP load balancer, the load balancer checks that IAP is enabled for the app or backend service.

If enabled, information about the protected resource, like the Google Cloud project number, the request URL, and any IAP credentials in the request headers or cookies, is sent to the IAP authentication server.

Next, **IAP** checks the user's browser credentials. If it doesn't find any credentials, the user is redirected to an OAuth 2.0 Google Account sign-in flow. A token is stored in a browser cookie for future sign-ins.

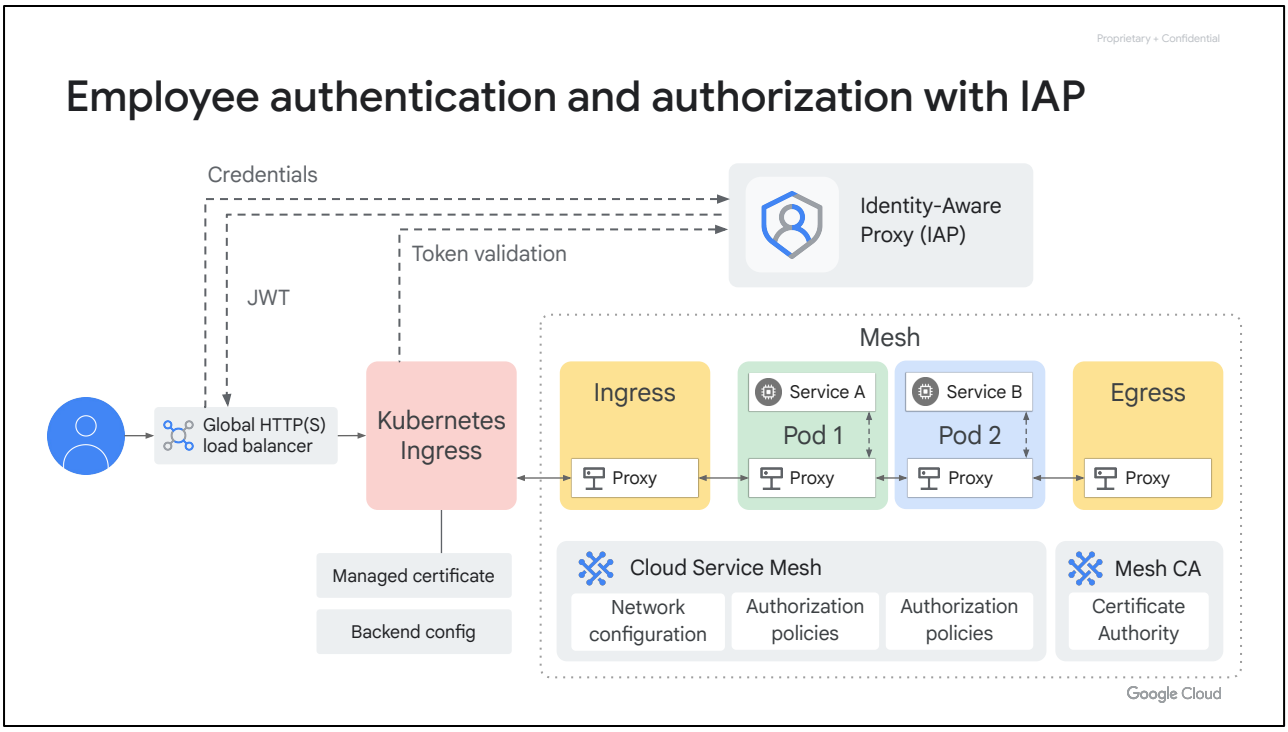
For existing users, you can use Google Cloud Directory Sync to synchronize with your Active Directory or LDAP server.

If credentials are valid, the authentication server uses those credentials to retrieve a user's identity (email address and user ID).

After authentication, IAP applies the relevant IAM policy to verify that the user is authorized to access the requested resource.

Users are authorized to use the application if they have the IAP-secured Web App User role applied on the project that contains the resource.

To manage the IAP-secured Web App User role list, use the IAP panel on the Google Cloud console.



Employee authentication is specifically tailored for employees within an organization.

In Google Cloud, you can leverage **Identity-Aware Proxy** (IAP) as the authentication and authorization provider for employees.

IAP allows access to internal applications without relying on network-level firewalls or a VPN.

IAP provides a centralized way to manage access to your HTTPS applications, and you can use group-based permissions to easily control which departments or teams have access to different applications.

When **IAP** is enabled for a resource, an OAuth 2.0 client ID and secret are automatically created. If you delete the automatically generated OAuth 2.0 credentials, IAP won't function correctly.

You can view and manage OAuth 2.0 credentials in the Google Cloud console APIs & services.

Proprietary + Confidential

Create an Ingress resource to configure load balancing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
  namespace: istio-system
  annotations:
  kubernetes.io/ingress.global-static-ip-name:
    static-ip.yaml
  kubernetes.io/managed-certificate: cert.yaml
spec:
  backend:
    serviceName:
      serviceName: istio-ingressgateway
      servicePort: 80
```

Google Cloud

Once IAP is set up and your application's OAuth 2.0 credentials are configured, you need to ensure your application is accessible via HTTPS. This is where Ingress resources come in. They help direct traffic to your application within the mesh.

You can create an **Ingress resource** to configure an HTTPS load balancer.

Google-managed SSL certificates are provisioned, renewed, and managed for your domain.

After the Ingress resource has been created, it forwards the request to the mesh ingress gateway, which then forwards the request to your app via the VirtualService configuration.

Proprietary + Confidential

RequestAuthentication configures IAP

```yaml
apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: ingressgateway-jwt-policy
  namespace: istio-system
spec:
  selector:
    matchLabels:
      app: istio-ingressgateway
  jwtRules:
  - issuer: “https://cloud.google.com/iap”
    jwksURI: ”https://...public_key-jwk”
  audiences:
  - $RCTOKEN_AUD
  fromHeaders:
  - name: ingress-authorization
    prefix: "Istio "
  outputPayloadToHeader: "verified-jwt"
  forwardOriginalToken: true
```

Google Cloud

RequestAuthentication configures IAP as the request authentication method.

By default, IAP issues a JSON Web Token (JWT) to verify user identity. This JWT is linked to the OAuth client you set up for your application.

For more granular control within Cloud Service Mesh, you can configure IAP to issue a special kind of JWT called a RequestContextToken (RCToken). The key advantage of an RCToken is that you can customize its "audience" field.

With RCToken, you can customize the "audience" of the JWT, allowing you to create more precise authorization rules in Cloud Service Mesh.

Proprietary + Confidential

Service Mesh Security

01 Authentication and encryption

02 Service authentication in the mesh

03 End-user authentication in Cloud Service Mesh

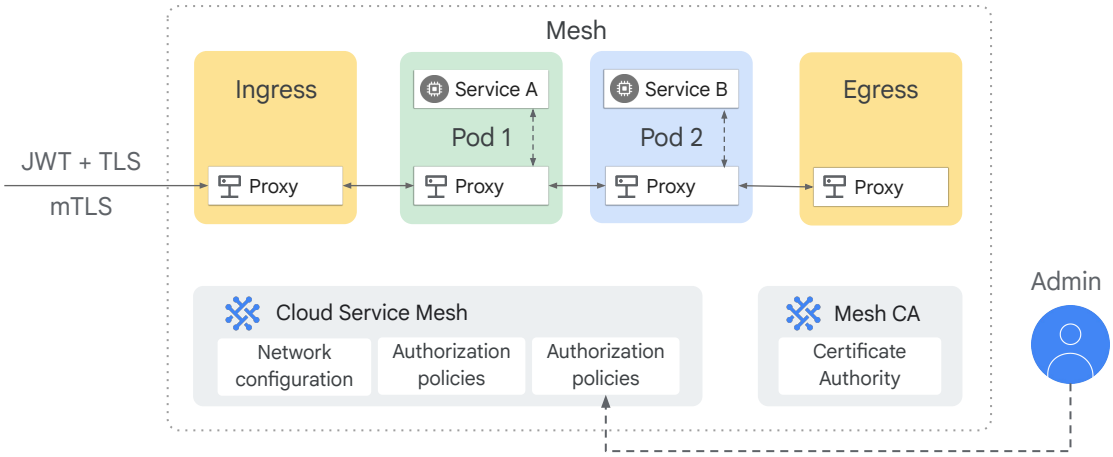
04 Authorization in Cloud Service Mesh

Google Cloud

Once a request has been authenticated, the information from the request can be used for authorization.

Proprietary + Confidential

Policy enforcement points help authorize requests



Google Cloud

Sidecar and perimeter proxies act as policy enforcement points (PEPs) and help authorize requests.

Operators specify mesh authorization policies using YAML files. When a proxy receives a request, it's evaluated against authorization policies by an authorization engine, and the request is either allowed or denied.

Proprietary + Confidential

Authorization policies

selector: Specifies the workloads affected by the policy.

action: Specifies whether to allow or deny requests.

rules: Specify when to trigger an action.

- from: Specifies sources of requests.
- to: Specifies the methods and paths allowed or denied.
- when: Specifies the conditions that would cause a request to match the rule.

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: httpbin
  namespace: foo
spec:
  selector:
    matchLabels:
      app: httpbin
action: ALLOW
rules:
- from:
  - source:
     principals:
["cluster.local/ns/default/sa/sleep"]
  to:
  - operation:
      methods: ["GET", “POST”]
      paths: ["/info”]
  when:
  - key: request.auth.claims[iss]
    values: ["https://accounts.google.com"]
```

Google Cloud

Authorization policies act as gatekeepers for your mesh workloads, controlling access between services as well as between end users and services.

They are defined using a single AuthorizationPolicy CRD, which provides flexible semantics with custom conditions and AUDIT, CUSTOM, DENY, and ALLOW actions.

These conditions are natively enforced by Envoy. Authorization policies support a range of protocols such as gRPC, HTTP, HTTPS, and HTTP/2 natively, and any TCP protocol.

An authorization policy includes a selector, an action, and a list of rules:

- The selector field specifies the workloads affected by the policy.
- The action field specifies whether to allow or deny requests.
- The rules specify when to trigger an action.
  - The *from* field in the rules specifies sources of requests.
  - The *to* field in the rules specifies the methods and paths allowed or denied. Notice that we apply these rules to layer 7 networking, while, in Kubernetes, we can only apply network policies at layer 4.
  - The *when* field specifies the conditions that would cause a request to match the rule.